

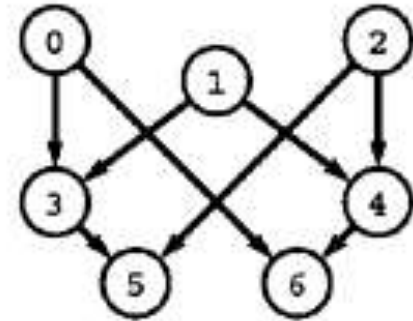
Maximum Flow (Part 3)

Reduction to Maximum Flow

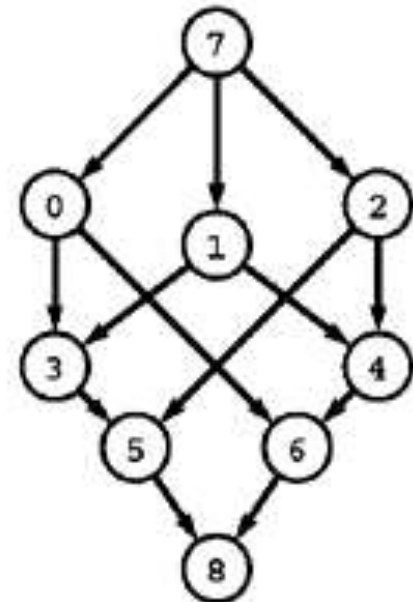
R1: Multiple sources and sinks

- Problem:
 - What if you have a problem with more than one source and more than one sink?
 - The network at the top has three sources (0, 1, and 2) and two sinks (5 and 6).
- Reduction: Create a network which
 - is a copy of the original network
 - with the addition of a new source 7 and
 - a new sink 8.
 - There is an edge from 7 to each original-network source with capacity equal to the sum of the capacities of that source's outgoing edges,
 - an edge from each original-network sink to 8 with capacity equal to the sum of the capacities of that sink's incoming edges.

	cap
0-3	2
0-6	3
1-3	3
1-4	1
2-4	1
2-5	1
3-5	2
4-6	3



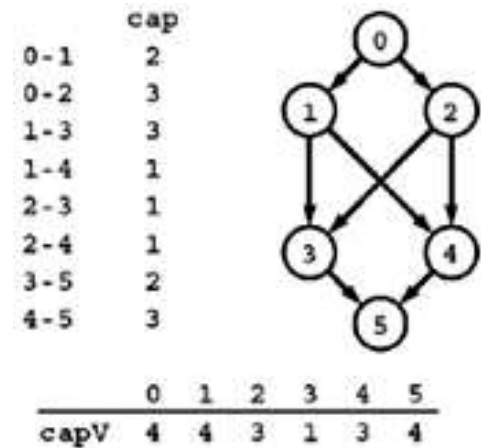
	cap
0-3	2
0-6	3
1-3	3
1-4	1
2-4	1
2-5	1
3-5	2
4-6	3
7-0	5
7-1	4
7-2	2
5-8	3
6-8	6



R2: Removing vertex capacities

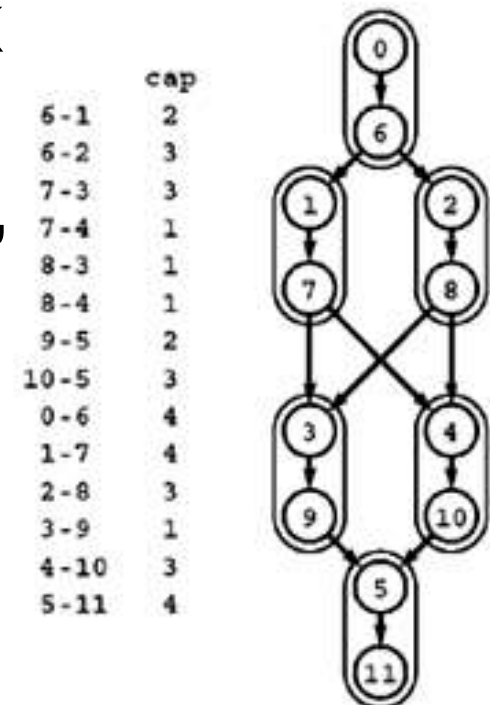
- Problem:

- Given a flow network, find a maxflow satisfying additional constraints specifying that the flow through each vertex must not exceed some fixed capacity.



- Reduction: Create a new network

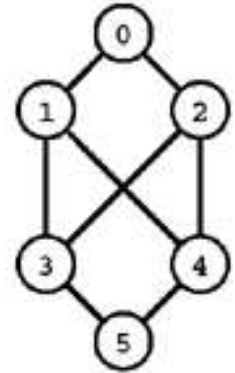
- Associate a new vertex u^* (where u^* denotes $u+V$) with each vertex u ,
- add an edge $u \rightarrow u^*$ whose capacity is the capacity of u ,
- include an edge $u^* \rightarrow v$ for each edge $u \rightarrow v$.



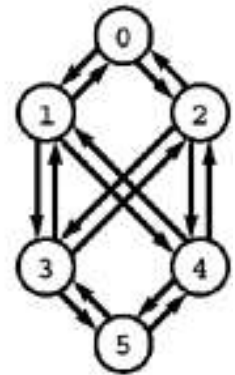
R4: Reduction from undirected networks

- Problem:
 - Given an undirected weighted graph find the maximum flow
- Reduction: Create a network s.t.
 - we can consider it to be a directed network with edges in each direction.

	cap
0-1	2
0-2	3
1-3	3
1-4	1
2-3	1
2-4	1
3-5	2
4-5	3

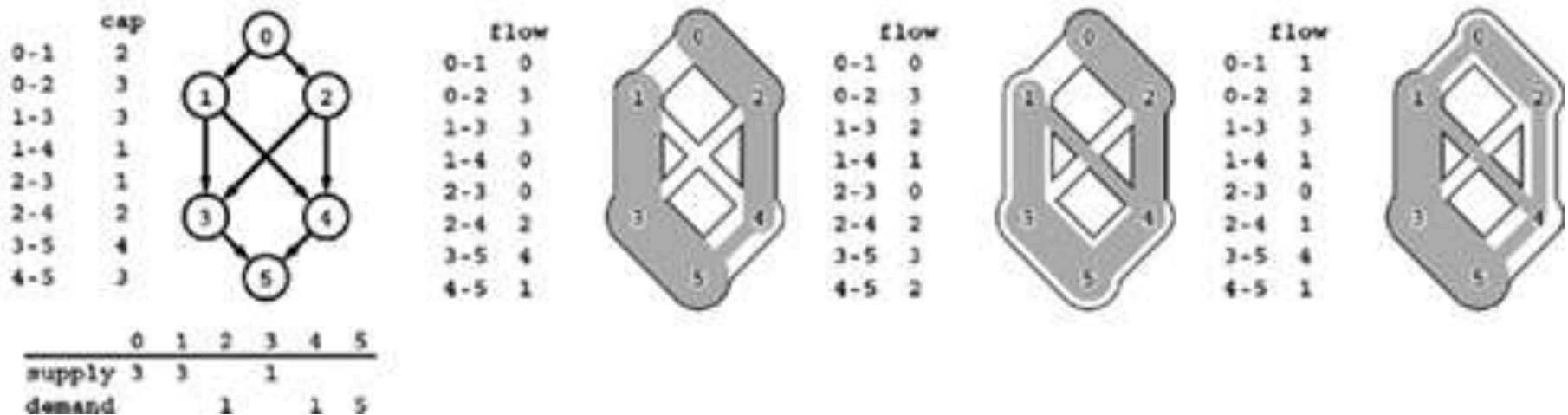


	cap
0-1	2
1-0	2
0-2	3
2-0	3
1-3	3
3-1	3
1-4	1
4-1	1
4-3	1
3-4	1
2-3	1
3-2	1
2-4	1
4-2	1
3-5	2
5-3	2
4-5	3
5-4	3



R5: Feasible flow

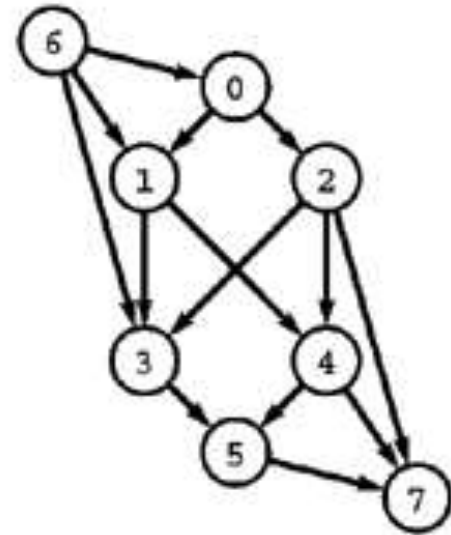
- Problem:
 - Suppose that a weight is assigned to each vertex in a flow network and is to be interpreted as supply (if positive) or demand (if negative), with the sum of the vertex weights equal to zero. Define a flow to be feasible if the difference between each vertex's outflow and inflow is equal to that vertex's weight (supply if positive and demand if negative). Given such a network, determine whether or not a feasible flow exists.



R5: Reduction from feasible solution

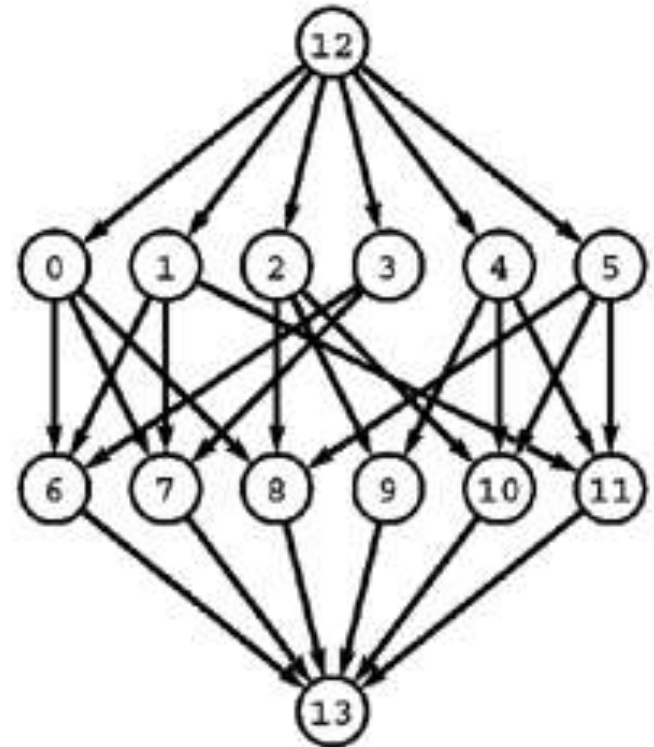
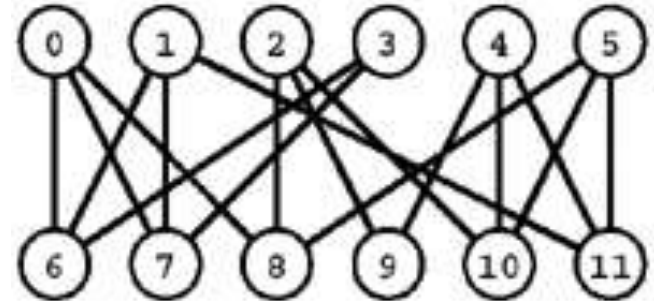
- Reduction: Create a network
 - by adding edges from a new source vertex to the supply vertices (each with capacity equal to the amount of the supply) and
 - edges to a new sink vertex from the demand vertices (each with capacity equal to the amount of the demand).
 - The network has a feasible flow if and only if this network has a flow (a maxflow) that fills all the edges from the sink and all the edges to the source.

	cap
0-1	2
0-2	3
1-3	3
1-4	1
2-3	1
2-4	1
3-5	2
4-5	3
6-0	3
6-1	3
6-3	1
2-7	1
4-7	1
5-7	5



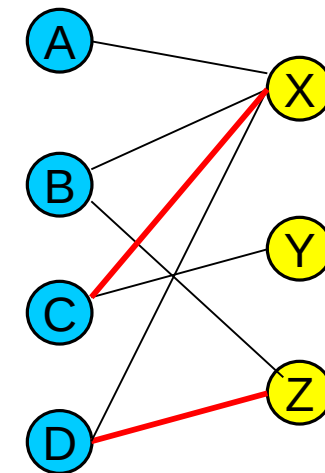
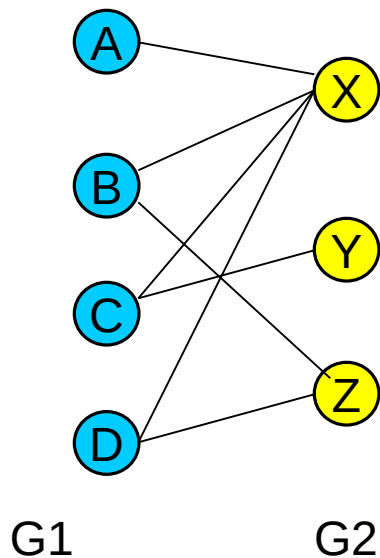
R6: Bipartite matching

- Problem:
 - Given a bipartite graph, find a set of edges of maximum cardinality such that each vertex is connected to at most one other vertex.
- Reduction: Construct an st-network
 - by directing all the edges from the top row to the bottom row,
 - adding a new source with edges to each vertex on the top row,
 - adding a new sink with edges to each vertex on the bottom row, and
 - assigning capacity 1 to all edges.
 - In any flow, at most one outgoing edge from each vertex on the top row can be filled and at most one incoming edge to each vertex on the bottom row can be filled, so a solution to the maxflow problem on this network gives a maximum matching for the bipartite graph.

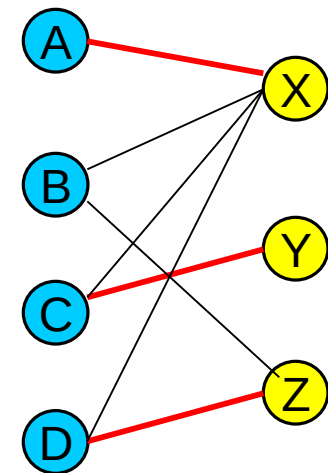


Model for Matching Problem

- Group1 on leftmost set, Group2 on rightmost set, edges if they are compatible



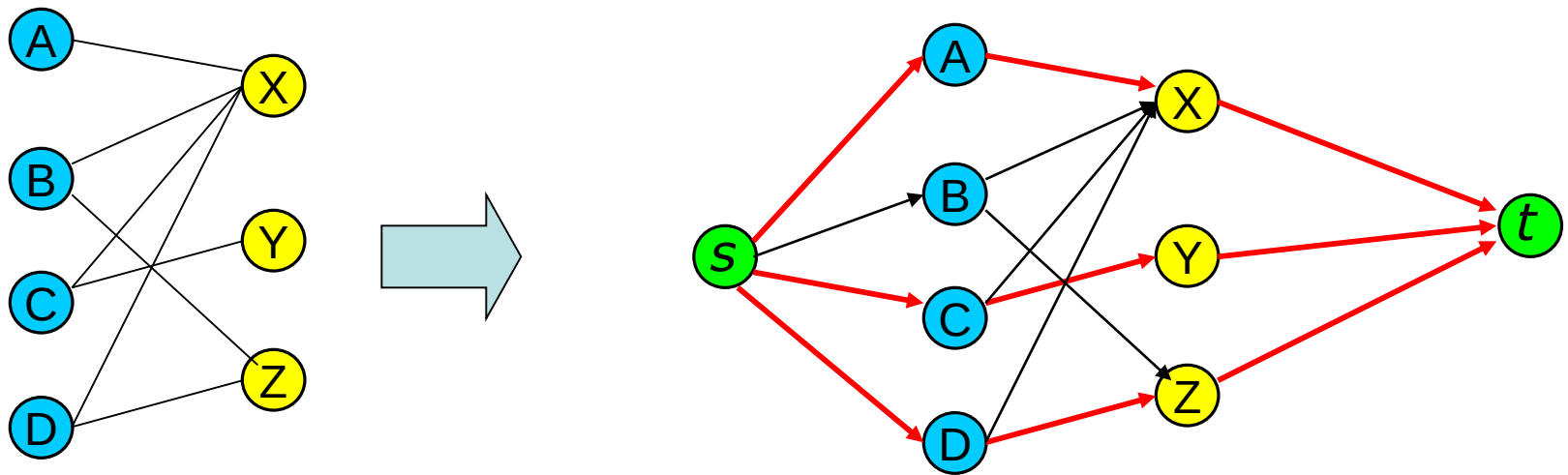
A matching



Optimal matching

Solution Using Max Flow

- Add a supersource, supersink, make each undirected edge directed with a flow of 1



Since the input is 1, flow conservation prevents multiple matchings

Edge Connectivity

- What is the minimum number of edges that need to be removed to separate a given graph into two pieces?
- Find a set of edges of minimal cardinality that does this separation.

Vertex Connectivity

- What is the minimum number of vertices that need to be removed to separate a given graph into two pieces?
- Try yourself